Study Report — SBMS (For exam/viva study)

Purpose: a concise, exam-friendly summary of the codebase, where CRUD logic lives, how data flows end-to-end, and short, paste-ready answers for viva prep.

1. Quick project map

- Root: utility scripts and SQL files (e.g., `populate_db.sql`, `setup_audit.sql`).
- `backend/` — Spring Boot app (Maven): controllers, services, repositories, entities, `application.properties`.
- `frontend/` — Next.js app (app router): `lib/api.ts` (API bridge), `app/` pages, `components/` UI.
- `docs/` — guides and reports (this file and `PROJECT_GUIDE.md`).

2. Core files to study (start here)

- `frontend/lib/api.ts` — central API bridge: all frontend CRUD calls.
- `frontend/app/admin/users/page.tsx` — representative admin UI using the API bridge.
- `backend/src/main/java/com/restroaudit/backend/controller/UserController.java` — CRUD endpoints for users.
- `backend/src/main/java/com/restroaudit/backend/service/OrderService.java` — complex business logic and transactions.
- `backend/src/main/resources/application.properties` — DB connection and JPA config.

3. Frontend: what to read and why

- `lib/api.ts`: shows `requestJson()` helper and exported functions like `fetchUsers()`, `registerUser()`, `createOrder()`; studying this shows how headers, JSON body, and error handling are handled.

Representative snippet (conceptual):

```
export async function requestJson<T>(path:string, init?: RequestInit){
  const res = await fetch(`${API_URL}${path}`, { ...init, headers: { 'Content-
Type':'application/json', ...(init?.headers||{}) }});
  if(!res.ok) throw new Error(await res.text());
  return res.json() as Promise<T>;
}
```

Tips: trace one flow end-to-end (e.g., create user) by following a page's handler → `lib/api.ts` → network call.

4. Backend: controllers → services → repositories

- Controllers: translate HTTP requests to Java objects and call services. Look for `@RestController` and mappings like `@PostMapping("/api/users")`.

- Services: contain business rules and `@Transactional` methods (important for order creation that updates stock and loyalty points).
- Repositories: `interface UserRepository extends JpaRepository<User, Long>` provides persistence.

Representative Controller snippet (conceptual):

```
@PostMapping("/api/users")
public ResponseEntity<?> createUser(@RequestBody User u){
  User saved = userService.save(u);
  return ResponseEntity.ok(saved);
}
```

Representative Service snippet (conceptual):

```
@Transactional
public Order saveOrder(Order order){
  // validate, persist order, update ingredients via recipe lookups, update
loyalty
  orderRepository.save(order);
  return order;
}
```

5. Database notes

- Config: `backend/src/main/resources/application.properties`

Example properties (dev):

```
spring.datasource.url=jdbc:mysql://localhost:3306/food_quality_db
spring.datasource.username=root
spring.datasource.password=<redact-for-report>
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Security note: do NOT include plaintext passwords in reports — replace with `DB_PASSWORD` environment variable in instructions.

6. End-to-end flow (Create Order) — step-by-step

1. User clicks confirm order in frontend UI → page calls `createOrder(payload)` from `lib/api.ts`.
2. `lib/api.ts` sends POST `/api/orders` with JSON and headers (including `X-Branch-Id`).
3. Spring controller receives JSON and maps to `Order` or DTO.
4. Controller calls `OrderService.saveOrder(orderDto)`.

5. `OrderService` runs transactional logic: validates items, looks up recipes/ingredients (uses `JdbcTemplate` for some lookups), saves `Order` & `OrderDetail` via repositories, deducts ingredient stock, increments loyalty points.
6. Database commit occurs at end of transaction; controller returns created order.
7. Frontend receives success, updates UI.

Key concepts to remember for viva: transactions (`@Transactional`), DTO vs Entity mapping, repository patterns, and error handling propagation back to the UI.

7. Paste-ready viva answers

- Q: How do you implement CRUD? — Frontend uses a single API bridge (`lib/api.ts`) that wraps `fetch`; backend exposes REST controllers that call services which use `JpaRepository` to persist entities.
- Q: How is consistency ensured when creating an order that affects stock? — `OrderService` runs in a `@Transactional` method so all DB updates (order + stock changes) commit or rollback together.
- Q: How are some lookup tables accessed? — Frequently-read lookup tables (e.g., `recipe`) may be accessed with `JdbcTemplate` when not modelled as an `@Entity`.

8. Run & dev commands (copyable)

Backend:

```
cd backend
mvn spring-boot:run
```

Frontend:

```
cd frontend
pnpm install
pnpm dev
# or
npm install
npm run dev
```

9. Short study checklist

- Read `frontend/lib/api.ts` and one page that uses it.
- Read `UserController` and `OrderService` (focus on transaction boundaries).
- Open `backend/src/main/resources/application.properties` and make sure you understand DB config.
- Run backend + frontend locally and do one create/read/update/delete to observe flows.

If you want, I can now:

- redact the DB password in `application.properties` and add an env-var example, or
- create a short `one-page cheat sheet` PDF from this markdown.

File created: `docs/STUDY_REPORT.md`